

PRODUCT REQUIREMENTS DOCUMENT

Question-Bank-Based Adaptive Competency Assessment



STARTER REPO

<https://github.com/MoHatemTC/round-2-masar-apdative-assessment>

Contents

Brief Description	3
Problem Statement	3
Goals	3
Users / Customers	4
List of Features (LoF)	4
Adaptive Engine — Detailed Behavior	5
Starting prior	5
Per-answer estimation (Bayesian posterior)	5
Convergence (stop conditions)	5
Bank exhaustion	6
Personalization (answer-invariant)	6
Question selection (difficulty-adaptive, type-varied)	6
Functional Requirements	6
Non-Functional Requirements	8
Deliverables	9

Brief Description

A web platform where an admin uploads a single question-bank JSON that fully defines competencies, sub-competencies, and questions (with metadata and difficulty), and the system turns it into an adaptive assessment. A candidate signs in, self-rates each competency 1–5, and an adaptive engine verifies their true level by asking a varied mix of bank questions (multiple-choice, open-ended/voice, coding, data analysis), stopping per competency once it is confident rather than asking a fixed number. Each question is personalized to the candidate's background from an optional CV, without changing what is tested or which answer is correct. Answers are auto-graded, a 1–5 level per competency is estimated with a Bayesian update and mapped to a 0–100% score across five named bands (Novice → Expert), and a report is produced and emailed.

Admins group uploads into reusable question sets, create assessments whose competencies are auto-derived from the chosen set, invite candidates by email, and review results and integrity signals. The intern builds this flow end-to-end: bank ingestion, the adaptive verification loop, grading, scoring, and a minimal admin + candidate UI. All 16 listed features are in scope for v1; the heaviest (browser audio capture + STT, sandboxed code execution) are bounded but not deferred.

Problem Statement

Fixed quizzes ask everyone the same questions and return a raw score that doesn't reflect a candidate's actual competency level or where they truly stand. Teams need an adaptive, competency-based assessment that verifies real skill efficiently from a reusable question bank and outputs an interpretable, personalized report.

Goals

- Ingest one JSON that fully defines competencies, sub-competencies, and questions (with metadata + difficulty) via an idempotent, validated import.
- Turn any uploaded bank into a reusable Question Set that drives assessment creation.
- Verify each competency's true 1–5 level with the fewest questions needed (adaptive convergence), not a fixed count.
- Personalize every question to the candidate's background without changing the tested skill or the correct answer.

- Auto-grade all question types (MCQ, open-ended/voice, coding, data-analysis) reliably and consistently.
- Produce an interpretable result: per-competency level plus an overall 0–100% mapped to five named bands.
- Deliver the result to the candidate and admin (report page + real email).
- Give admins a simple flow: upload bank → create assessment (competencies auto-derived) → invite → review.
- Keep the system observable and reproducible: every AI call and grade is logged.
- Ship it deployable end-to-end (DB migrations, backend API, minimal UI).

Users / Customers

- Hiring / L&D admins — build assessments from a bank and review candidate results.
- Candidates — take the adaptive assessment and receive a personalized report.
- Internal reviewers / engineers — audit question quality, grading, and integrity signals.

List of Features (LoF)

1. JSON question-bank import (competencies + sub-competencies + questions + difficulty).
2. Per-question-type payload validation with clear, row-level errors.
3. Upload → auto-created reusable Question Set.
4. Question Bank browser (filter by type / competency / difficulty).
5. Competency and sub-competency management.
6. Set-driven assessment creation (competencies auto-derived from the chosen set).
7. Candidate self-rating intake (1–5 per competency).
8. Optional CV upload → per-competency initial estimate.
9. Adaptive verification loop (per competency, converges automatically).
10. Difficulty-adaptive + type-varied question selection.
11. Question personalization to the candidate's background (answer-invariant).
12. Multi-format questions — four tool types: MCQ, coding, voice (open-ended, audio), and data-analysis/visualization (voice and data-analysis are both graded by rubric).

- 13. Auto-grading per type (MCQ key match; coding sandboxed test cases + LLM judge; open-ended/voice/data-analysis by rubric/LLM judge).
- 14. Bayesian 1–5 level estimation → 0–100% + five-band scoring.
- 15. Candidate report + admin review dashboard.
- 16. Email delivery (invite + report) with send logs.

Adaptive Engine — Detailed Behavior

Starting prior

For each measured competency, form the starting 1–5 belief by blending the optional CV estimate and the candidate self-rating with equal weight:

```
prior_level = round( 0.5 × cv_estimate + 0.5 × self_rating )
```

If no CV is provided, `prior_level = self_rating`. If neither is available, default to 3. The prior starts at low confidence (it is a belief, not a measurement).

Per-answer estimation (Bayesian posterior)

Represent each competency's level as a probability distribution over {1,2,3,4,5}. Initialize the distribution from the prior (peaked on `prior_level`). After each graded answer:

1. Convert the graded score (0–5) and the question difficulty into a likelihood over levels — a high score on a hard question shifts mass upward; a low score on an easy question shifts it down.
2. Multiply the prior distribution by the likelihood and renormalize to get the posterior.
3. `level = argmax(posterior)`; `confidence = 1 – normalized_spread(posterior)` (a tight distribution → high confidence).
4. Clamp confidence to the anti-early-stop ceiling below, then record it in `level_history`.

The update is deterministic and requires no LLM call, so it is cheap, reproducible, and debuggable. The self-rating and CV enter only through the prior, not repeatedly.

Convergence (stop conditions)

A competency is done when ANY of these fire:

- `confidence ≥ CONFIDENCE_TARGET` (0.90), OR

- the same level is estimated STABLE_WINDOW (3) times in a row, OR
- MAX_QUESTIONS (10) have been asked for that competency.

Anti-early-stop ceiling: confidence is capped by question count {1: 0.50, 2: 0.70, 3: 0.85}, and 0.97 thereafter — so one strong answer cannot trigger the confidence stop; the engine must probe at least ~4 questions before it can converge on confidence.

Bank exhaustion

If a competency's bank runs out before it converges, the engine generates additional open-ended questions (never MCQ — an invented answer key can't be trusted) and keeps probing until it converges or hits MAX_QUESTIONS. The final level is always recorded; if it converged only by hitting the cap at low confidence, the report flags it as low-confidence.

Personalization (answer-invariant)

Before serving, an LLM rewrites the question stem/context AND may reword MCQ options to reference the candidate's background. **Hard rule:** the rewrite MUST NOT change what skill is tested and MUST keep the correct answer identical — the server re-injects the original answer_key / test_cases / criteria after the rewrite so the measured competency stays deterministic. Never send answer-bearing fields to the browser.

Question selection (difficulty-adaptive, type-varied)

- **Difficulty:** always target the current level estimate — pick a bank question whose difficulty $\approx$ round(current level estimate). As the estimate moves, the difficulty of served questions moves with it.
- **Type variety:** among candidate questions at the target difficulty, prefer the least-asked tool_type so the candidate sees a mix (MCQ, open-ended/voice, coding, data analysis).
- Never repeat a question already served in the session.

Functional Requirements

- The system SHALL accept a JSON array where each item defines track, sub_competency, tool_type, difficulty, body, and payload.
- Import SHALL upsert competencies by code and questions by source_ref (idempotent — re-import never duplicates).

- Import SHALL validate each question's payload against its tool_type and return per-row errors on failure.
- Import SHALL create or update a named Question Set containing all imported questions.
- The system SHALL derive an assessment's measured competencies from the chosen Question Set.
- The system SHALL collect a 1–5 self-rating for each measured competency during intake.
- The system SHALL optionally accept a CV and derive an initial 1–5 estimate per competency from it.
- The engine SHALL blend the CV estimate and self-rating with equal weight into a starting prior per competency: $\text{prior} = \text{round}(0.5 \cdot \text{CV} + 0.5 \cdot \text{self})$; self-rating alone when no CV; default 3.
- The engine SHALL maintain a Bayesian posterior distribution over levels {1..5} per competency and update it after each graded answer using the score and the question's difficulty.
- The engine SHALL derive $\text{level} = \text{argmax}(\text{posterior})$ and $\text{confidence} = 1 - \text{normalized spread}$ after each update.
- The engine SHALL select the next question at a difficulty targeting the current level estimate, and among those prefer the least-asked tool type.
- The engine SHALL personalize the selected question (stem and MCQ options) without changing the tested skill, and SHALL re-inject the original answer key so the correct answer is invariant.
- The engine SHALL grade MCQ by exact key match, coding by sandboxed test cases plus an LLM judge on approach, and open-ended/voice/data-analysis by rubric (LLM judge against evaluation_criteria/expected_insights); voice answers SHALL be captured as audio and transcribed before rubric grading.
- The engine SHALL treat difficulty on a 1–5 level scale, mapping the bank's easy/medium/hard via a fixed table (easy=2, medium=3, hard=4) for both selection and estimation.
- The engine SHALL cap confidence by question count {1:0.50, 2:0.70, 3:0.85, else 0.97} so a single answer cannot trigger the confidence stop.
- The engine SHALL stop a competency at confidence ≥ 0.90 , OR the same level 3 times in a row, OR 10 questions asked.
- The engine SHALL, when a competency's bank is exhausted before convergence, generate additional open-ended questions until it converges or hits the max cap.

- The system SHALL map each verified level to a per-competency score ($\text{pct} = \text{level} \times 20$) and one of five named bands, and flag low-confidence results on the report.
- The system SHALL compute the overall score as the mean of the per-competency percentages ($\text{overall_pct} = \text{mean}(\text{level} \times 20)$) and derive the overall band from `overall_pct`.
- The system SHALL persist each served question, answer, score, and grading rationale.
- The system SHALL generate a candidate report (per-competency level + overall % / band).
- The system SHALL email the report to the candidate and notify the admin, and persist a send log for each email.
- Admins SHALL invite candidates by email and see taken / not-taken status.
- The system SHALL never expose answer keys to the browser before grading.

Non-Functional Requirements

- A candidate turn (serve a question or grade an answer) SHALL respond within ~5s at p95.
- The engine SHALL be resumable: state persists so a page reload never loses progress or double-counts an answer (unique (session, question_number)).
- Imports SHALL be idempotent and safe to re-run.
- All LLM prompts, responses, and grades SHALL be logged for reproducibility and audit.
- The system SHALL degrade gracefully when an external service (LLM / email / STT) fails.
- Coding submissions SHALL run in a sandboxed, resource-bounded environment; untrusted code never executes on the app host.
- Secrets SHALL live in environment variables — never in code or the repository.
- The UI SHALL be responsive and support light and dark themes.
- Database changes SHALL ship as ordered, additive migrations.
- The system SHALL bound cost (max questions per competency, capped tokens per call, Bayesian estimate needs no LLM).
- PII (CV, audio, contact details) SHALL be access-controlled and never publicly exposed.

Deliverables

- Database schema + migrations (competencies, question_bank, question_sets/items, assessments, sessions, answers, final_reports, session_competency_results, email_logs, ai_logs).
- Question-bank import API with validation + automatic Question Set creation.
- Per-type question schema + validator (single source of truth).
- Adaptive verification engine (state machine: init → ask → grade → estimate → converge → finalize) with the Bayesian estimator and difficulty-adaptive selection.
- Grading services (MCQ key match; open-ended/voice/data-analysis rubric, with STT for voice; coding sandbox + LLM judge).
- Scoring module (1–5 → 0–100% + five bands, low-confidence flag) + report generation.
- Minimal admin UI (import bank, browse bank, create set-driven assessment, invite, review results).
- Minimal candidate UI (self-rating intake → optional CV → adaptive Q&A incl. audio capture → report).
- Email integration (invite + report) with send logs.
- README + architecture doc + seed question bank + a demo script.